

Tema 1 - Tecnologías móviles.

Fundamentos de Android

Índice

1. INTRODUCCIÓN	2
1.1. CARACTERÍSTICAS DE LOS DISPOSITIVOS MÓVILES	2
1.2. TIPOS DE DISPOSITIVOS MÓVILES	3
1.3. LIMITACIONES DE LOS DISPOSITIVOS MÓVILES	4
2. SISTEMAS OPERATIVOS MÓVILES	5
2.1. ANDROID.....	6
2.2. IOS	7
2.3. OTROS SO PARA DISPOSITIVOS MÓVILES	8
3. NOCIONES PREVIAS AL DESARROLLO DE APLICACIONES MÓVILES	9
3.1. DESARROLLO NATIVO	9
3.1.1. <i>Desarrollo multiplataforma compilado a nativo</i>	<i>10</i>
3.2. ENTORNOS DE DESARROLLO INTEGRADO DE APLICACIONES MÓVILES	10
3.2.1. <i>Ios</i>	<i>10</i>
3.2.2. <i>Android.....</i>	<i>11</i>
4. DESARROLLO DE APLICACIONES ANDROID	12
4.1. COMPONENTES DE UNA APLICACIÓN	12
4.2. ANDROID MANIFEST	13
4.3. RECURSOS DE UNA APLICACIÓN	14
4.4. LA CLASE R.....	15
4.5. ACTIVIDADES.....	16
4.5.1. <i>Ciclo de vida de una actividad.....</i>	<i>17</i>
4.6. INTENTS Y FILTROS	19
4.6.1. <i>Intent explícito.....</i>	<i>20</i>
4.6.2. <i>Intent implícitos.....</i>	<i>20</i>
4.6.3. <i>Elementos de un Intent</i>	<i>21</i>

1. Introducción

De entre la fugaz y vertiginosa sucesión de dispositivos informáticos y electrónicos que han aparecido en la sociedad moderna, ninguno de ellos ha producido una variación tan radical en la mentalidad y conducta popular como la adopción de los **smartphones** y las **redes móviles**; especialmente, la conjugación entre ambas, capaces de simplificar en un solo dispositivo multitud de tareas para las que hasta ahora necesitábamos variados y complejos aparatos.

Antes de profundizar en el desarrollo de aplicaciones móviles, es conveniente conocer las tecnologías móviles que pueden encontrarse en el mercado, dispositivos que las soportan y aquellos otros aspectos que pueden ser relevantes a la hora de programar una aplicación móvil.

1.1. Características de los dispositivos móviles

Una primera definición del concepto de **dispositivo móvil** sería describirlo como aquel aparato con capacidad de procesado y almacenamiento, de pequeño tamaño, portable, autonomía de funcionamiento en cuanto a batería, conexión permanente o semipermanente a una red de comunicaciones y con variadas funciones. Esta definición permite realizar una agrupación inicial en cuatro grandes bloques:

1. **Capacidad de procesado:** refiere a la capacidad de cálculo (en forma de velocidad de procesado) y almacenamiento, que incluiría el necesario para el procesado y la persistencia de la información depositada. En virtud de la relevancia de esta capacidad, se denomina a los dispositivos móviles *microordenadores*.
2. **Tamaño:** el concepto de portabilidad se lleva a sus máximos extremos, alejándose de los ordenadores portátiles, y formando parte de la vida diaria del usuario independientemente de la actividad que se éste realizando.
3. **Movilidad:** asociada a la anterior característica, pero también a la no dependencia de cableado para su alimentación o comunicación. Es decir, pequeño tamaño, batería duradera y comunicación inalámbrica son los tres pilares sobre los que se asienta esta propiedad.

4. **Conectividad:** si bien gran parte de las tecnologías aplicadas a la conexión de dispositivos móviles ha quedado obsoleta, los actuales avances en redes móviles y la masiva incorporación del uso de redes de datos han creado un nuevo paradigma a la hora de definir estos dispositivos y su uso.

Además de estas propiedades, se pueden incluir muchas otras, tales como como sistema operativo, diseño, ergonomía, tamaño de pantalla, gadgets incorporados...

1.2. Tipos de dispositivos móviles

Teniendo en cuenta las características descritas anteriormente y con el objetivo de realizar una clasificación preliminar, se pueden agrupar los dispositivos móviles en los siguientes tipos:

- **Teléfonos móviles:** incluimos en este apartado aquellos teléfonos cuya funcionalidad se limita básicamente a la comunicación por voz, si bien en algunos casos pueden incluir operatividad SMS, cámara fotográfica, agenda.
- **Handhelds:** con variado diseño, inicialmente con pantalla plegable sobre el teclado, fueron utilizados como agenda/organizador con posibilidad de ejecutar algunas aplicaciones (científicas o financieras).
- **Netbooks:** lanzados al mercado en 1999 por Psion y con buena acogida hasta 2007, se diseñaron originariamente como ordenadores compactos de pequeño tamaño y bajo precio con suficiente poder de cálculo como para darle uso ofimático y tener acceso a internet.
- **E-Book Readers:** con variadas denominaciones (libro electrónico, eBook, libro digital), son dispositivos usados para leer ficheros de documentos electrónicos. Suelen ser ligeros, con alta capacidad de almacenaje, bajo consumo y tecnología de pantalla tinta electrónica o *e-Ink*.
- **Tablets:** con tamaño intermedio entre el ordenador y el móvil, tienen como principales características la ligereza, autonomía y fácil manejo a través de su pantalla táctil. Se considera que su irrupción en el mercado data de principios de este siglo, y su popularización se debe en gran medida al iPad de Apple.
- **Personal digital assistant (PDA):** son dispositivos que intentan combinar variadas prestaciones con la pretensión de funcionar como un organizador digital.

Inicialmente manejados con un lápiz (stylus), decayeron con la eclosión en el mercado de los teléfonos móviles inteligentes.

- **Smartphones:** dispositivos cuya principal funcionalidad es la de teléfono móvil, pero que por sus características (similares a las de un ordenador personal) y la variedad de recursos electrónicos que progresivamente se le han ido añadiendo, han hecho que la mayoría de los usuarios recurran a ellos para muchas más funcionalidades que simplemente la comunicación telefónica. Suelen ser características comunes a todos ellos las pantallas de alta calidad, la múltiple conectividad, la gran capacidad de procesamiento y almacenaje, y la riqueza en dispositivos incorporados (cámara, sensores, GPS).



Figura 1. Línea de tiempo de dispositivos móviles.

1.3. Limitaciones de los dispositivos móviles

La programación de aplicaciones para móviles tiene muchos aspectos similares al desarrollo de aplicaciones para escritorio, pero es necesario entender que para conseguir que los programas funcionen correctamente en un amplio y variado número de dispositivos se debe conocer cuáles son las **limitaciones** o restricciones a las que estará sujeto el desarrollo.

La mayoría de las limitaciones estarán relacionadas con las características **hardware**. Aunque el mercado ha evolucionado de manera importante, se han de tener en cuenta las variaciones que se pueden encontrar en la capacidad de procesado y almacenamiento, evitando, en la medida que sea posible, la excesiva carga en elementos multimedia.

Un buen punto de partida es **evitar prototipar para dispositivos de alta gama**: si se logra que una aplicación funcione bien en teléfonos con pocos recursos, seguramente lo hará aún mejor en otros de gama superior. Es aconsejable, en cualquier caso, optimizar el uso del procesador para una rápida ejecución del código y respuesta de la aplicación.

La diversidad de **tamaño de pantalla** también puede ser relevante, ya que diferentes resoluciones harán obtener resultados muy lejanos a los deseados por el programador. Además, no todos los equipos están dotados de **sensores** y las tecnologías que son consideradas básicas, lo que puede provocar que ni siquiera admitan la instalación de la aplicación.

Es importante conocer las diferencias entre **plataformas operativas** y las versiones que existan de ellas, ya que nunca se tendrá garantizada la retrocompatibilidad u operatividad futura del software desarrollado. También hay que tener cuidado al incorporar herramientas y librerías de terceras partes, puesto que no son admitidas por todos los dispositivos.

Por último, a pesar de la generalización de las redes móviles y con ella la conectividad de casi la totalidad de los dispositivos, las posibles **limitaciones del ancho de banda** y las desconexiones temporales de la red por falta de cobertura pueden llevar al fracaso de una aplicación si su lógica demanda una conexión constante a la red.

Por todo lo anterior, es recomendable que antes de llevar una aplicación a la fase de explotación, se pruebe y verifique en el más variado número de dispositivos posibles. En caso de no poder hacerlo con equipos reales, deberán emplearse los emuladores que proporcionan los entornos de desarrollo.

2. Sistemas operativos móviles

Al igual que en los ordenadores personales, el **sistema operativo móvil** es el conjunto de programas de bajo nivel que permiten la abstracción de las características propias del hardware y provee servicios a las aplicaciones que se ejecutan sobre él. La mayoría de los sistemas operativos móviles están basados en el modelo de capas con variada

complejidad y multiplicidad a medida que se alejan de los elementos hardware. Simplificando cada una de ellas, podemos encontrarlas siguientes:

- **Núcleo o *kernel*.** Proporciona el acceso a los elementos del hardware, ofreciendo servicios a las capas superiores a través de los controladores o drivers, la gestión de procesos el sistema de archivos y la gestión de la memoria.
- ***Middleware*.** Conjunto de módulos que posibilitan la existencia de aplicaciones. Es transparente para el usuario y ofrece servicios como el motor de mensajería y comunicaciones, códec multimedia, intérpretes web, gestión del dispositivo y seguridad.
- **Entorno de ejecución de aplicaciones.** Gestor de aplicaciones e interfaces abierto que permite la programación por parte de los desarrolladores para la creación de software.
- **Interfaces de usuario.** Facilitan la relación con el usuario y se encargan de la presentación visual de la aplicación. Incluyen los componentes gráficos (botones, pantallas, listas, etc.) y el marco de interacción.
- **Aplicaciones nativas.** Propias de cada uno de los modelos y fabricantes.

A continuación, se describen los sistemas operativos más relevantes en la actualidad.

2.1. Android

Lanzado en 2007, es el sistema operativo líder en el mercado móvil. Fue desarrollado por la Open Handset Alliance (OHA), una agrupación de 78 compañías lideradas por **Google**. Su principal cualidad es su **carácter abierto**, ya que se distribuye bajo dos tipos de licencias: una que abarca el código del kernel (*GNU GPLv2*) y otra para el resto de los componentes del sistema (licencia bajo *APACHE v2*).

La arquitectura de Android tiene las siguientes **capas**:

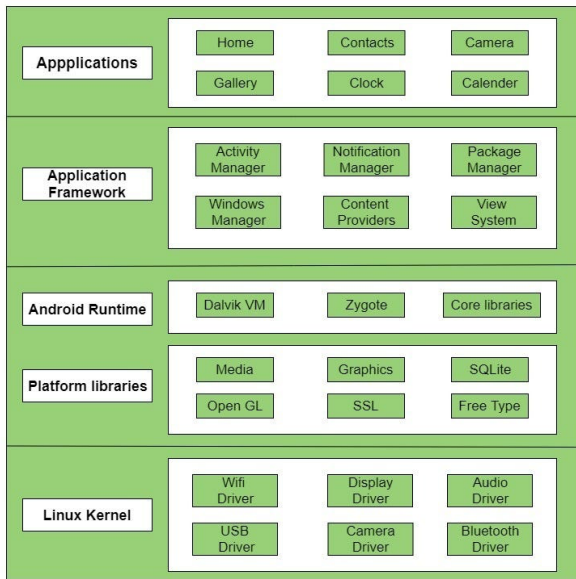


Figura 2. Arquitectura en capas de Android.

a) **Kernel de Linux.** El *runtime* de Android (*ART*) se basa en el kernel de Linux, con funcionalidades como la generación de subprocesos y la administración de memoria de bajo nivel.

b) **Capa de abstracción de hardware (HAL).** Proporciona interfaces que enlazan las capacidades de hardware del dispositivo al framework de nivel más alto. Posee varias bibliotecas, con una interfaz para cada tipo de componente (cámara, bluetooth...).

c) **Runtime.** Cada app ejecuta sus propios procesos con sus propias instancias del ART, ya que está diseñado para ejecutar varias máquinas virtuales en dispositivos de memoria baja ejecutando archivos *DEX*, un formato de código de bytes específico de Android, y optimizado para ocupar un espacio de memoria mínimo.

- d) **Bibliotecas C/C++ nativas.** Muchos componentes y servicios se basan en código nativo, que requiere bibliotecas nativas escritas en C y C++. La plataforma Android proporciona la API para posibilitar la funcionalidad de estas bibliotecas.
- e) **Framework de la Java API.** Todo el conjunto de funciones del SO Android está disponible mediante API escritas en el lenguaje Java.
- f) **Apps del sistema.** Android incluye un conjunto de apps nativas para correo electrónico mensajería SMS, calendarios, navegación en Internet.

2.2. iOS

Desarrollado por **Apple** y originariamente denominado iPhoneOS, es un derivado de MacOS X. Su principal fuerte es la **combinación**, casi perfecta, **entre hardware y software**, así como el manejo de la pantalla multitáctil que lo convierte en un sistema operativo basado en la manipulación directa. Su arquitectura también está basada en capas: las capas más altas contienen los servicios y tecnologías más importantes para el desarrollo de aplicaciones y las capas más bajas controlan los servicios básicos.

a) **Cocoa Touch:** capa superior y más importante para el desarrollo de aplicaciones iOS. Es la que los usuarios utilizan para interactuar con las aplicaciones, es decir, la capa visible. Esta capa está formada fundamentalmente por dos frameworks: *UIKit* (clases para el desarrollo de una interfaz de usuario) y *Foundation Framework* (acceso y manejo de objetos, servicios del SO).

b) **Media Services:** provee los servicios de audio, gráficos y multimedia a la capa superior.

c) **Core Services:** proporciona los servicios imprescindibles del sistema para poder ser utilizados por todas las aplicaciones (base de datos, acceso a la red).

d) **Core OS:** núcleo del sistema con las características de bajo nivel (manejo de memoria, seguridad, drivers del dispositivo).

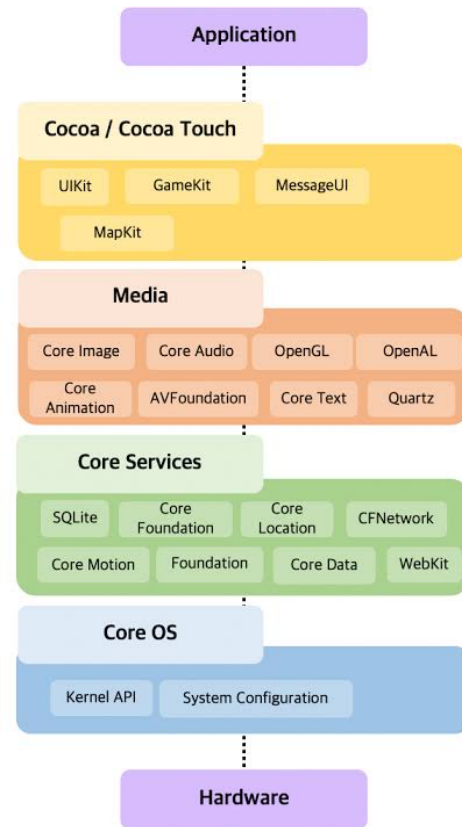


Figura 3. Arquitectura en capas de iOS.

2.3. Otros SO para dispositivos móviles

En el breve espacio de tiempo que han ocupado los dispositivos móviles en el mundo tecnológico, otros sistemas operativos han competido fugazmente con Android e iOS. Aunque su relevancia es mínima en la cuota de mercado actual, algunos han marcado tendencias en las funcionalidades a implementar en un SO para dispositivos móviles.

En primer lugar, **SymbianOS** dominó el panorama de la telefonía móvil durante la década de 2000, abanderado principalmente por Nokia y su interfaz S60. Diseñado en C++ para aprovechar un hardware con recursos muy limitados, este sistema sentó las bases del concepto de teléfono inteligente al popularizar la multitarea, la conectividad por paquetes de datos y la distribución de aplicaciones de terceros mediante paquetes instalables o la plataforma Java ME.

Por su parte, **BlackBerryOS** se estableció principalmente en el entorno empresarial gracias a su integración corporativa con servidores propietarios y a su teclado físico. Este OS definió los primeros estándares rigurosos de cifrado de extremo a extremo y separación entre perfiles laborales y personales. Además, popularizó las notificaciones push en tiempo real y la mensajería instantánea antes de la llegada de aplicaciones dedicadas como WhatsApp o Telegram.

Microsoft también intentó introducirse al mercado móvil con **Windows Phone**, comercializado posteriormente como Windows 10 Mobile. Para ello, estableció un ecosistema basado en C#, XAML y la plataforma .NET. Su mayor repercusión tecnológica y visual fue la aplicación del lenguaje tipográfico Metro UI y sus Live Tiles, lo que obligó al resto de la industria a adoptar el diseño plano que derivaría en iOS 7 y en Material Design de Google.

Finalmente, **Ubuntu Touch** surgió de la mano de Canonical con el objetivo de trasladar una distribución completa de GNU/Linux al ecosistema móvil mediante Qt, QML y el servidor gráfico Wayland. Fue pionero en una experiencia de usuario guiada exclusivamente por gestos desde los bordes de la pantalla, un paradigma de interacción que la tecnología móvil contemporánea terminó asimilando de forma generalizada.

3. Nociones previas al desarrollo de aplicaciones móviles

El mundo de desarrollo de aplicaciones para móviles dispone de infinidad de herramientas, lenguajes y entornos para elegir antes de comenzar a escribir código.

3.1. Desarrollo nativo

A pesar de que cada plataforma (iOS, Android...) utiliza lenguajes y paradigmas de programación diferentes y herramientas propias, el desarrollo nativo garantiza una gran flexibilidad, adaptación total al entorno y un máximo rendimiento.

La principal desventaja del desarrollo nativo deriva de hacer que una aplicación sea compatible con más de un entorno: se tendrá que dominar más de un lenguaje y distintas

herramientas, lo que aumentará el tiempo de desarrollo. Por ello, generalmente, los programadores se suelen especializar en una plataforma, sacándole el máximo rendimiento a la programación en la misma.

3.1.1. Desarrollo multiplataforma compilado a nativo

Una opción intermedia es utilizar plataformas mixtas que permitan independizar el desarrollo del lenguaje nativo de cada plataforma móvil. Con un único lenguaje, se podrán crear aplicaciones para todos los sistemas operativos, aunque siempre habrá que realizar pequeñas adaptaciones.

La más conocida es *Xamarin*, basada en el lenguaje C# de Microsoft y en la plataforma .NET, que gracias a sus herramientas permite crear aplicaciones multiplataforma, reutilizando gran parte del código (a excepción de la interfaz). Actualmente, el soporte oficial de Xamarin se ha descontinuado en favor de *.NET MAUI*.

Las aplicaciones escritas con estas plataformas mixtas se compilan a código nativo en cada plataforma específica, lo que le da el mismo rendimiento que el de una aplicación nativa. También tiene acceso directo nativo a todas las API de cada plataforma, así como a los controles de interfaz de usuario nativos.

3.2. Entornos de desarrollo integrado de aplicaciones móviles

A continuación, se destacan los lenguajes e IDEs específicos para el desarrollo nativo de las plataformas clave de desarrollo móvil.

3.2.1. iOS

iOS emplea **Swift** como lenguaje de programación. Se trata de un innovador lenguaje de programación para Cocoa y Cocoa Touch con mejoras sobre Objective-C, que es mucho menos transigente con los errores y tiene una sintaxis más complicada.

La combinación de Swift con **Xcode**, el IDE oficial de MacOS, facilita y agiliza el desarrollo para esta plataforma, además de poseer un gestor de interfaces muy intuitivo (Interface Builder, una herencia de NeXT) y buenas herramientas de depuración.

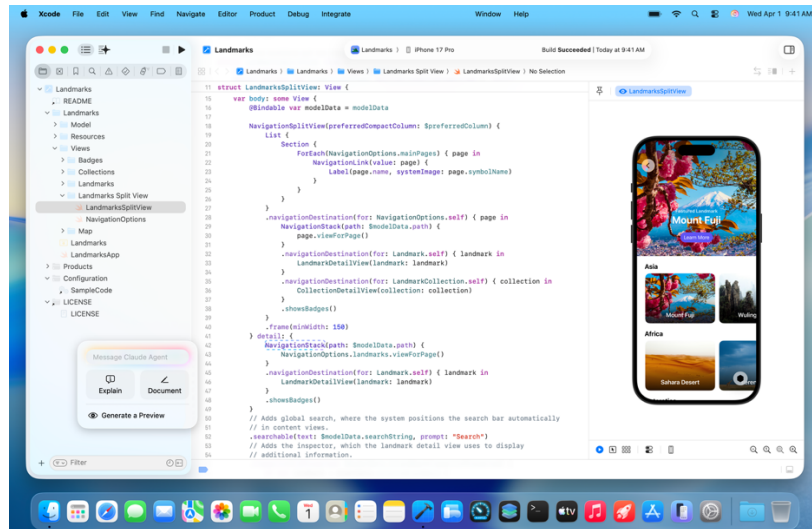


Figura 4. Captura de pantalla de xCode, IDE para iOS, en funcionamiento.

3.2.2. Android

Kotlin es el lenguaje oficial y preferente para el desarrollo nativo en Android, sustituyendo progresivamente el rol central que históricamente tuvo Java (con el cual todavía mantiene una interoperabilidad total del 100%). Este lenguaje desarrollado por Google es moderno, conciso, seguro frente a nulos y altamente expresivo. Está optimizado tanto para interactuar con las APIs del sistema como para gestionar tareas asíncronas complejas mediante corrutinas.

En cuanto a la creación de interfaces de usuario, el estándar moderno es **Jetpack Compose**, un toolkit declarativo basado íntegramente en Kotlin. No obstante, tradicionalmente se han empleado interfaces basadas en XML en aplicaciones escritas en Java; de hecho, XML sigue siendo relevante para la definición de recursos de la aplicación (cadenas de texto, dimensiones, colores o vectores gráficos), así como para el mantenimiento del sistema tradicional de vistas (*layouts*).

El entorno de desarrollo oficial es **Android Studio**, un IDE desarrollado por Google sobre la plataforma IntelliJ IDEA de JetBrains. Ofrece un conjunto integral de herramientas para el desarrollo, perfilado de rendimiento y depuración de software, apoyado en el

motor de compilación **Gradle** y en la simulación de terminales mediante **AVD** (*Android Virtual Devices*).

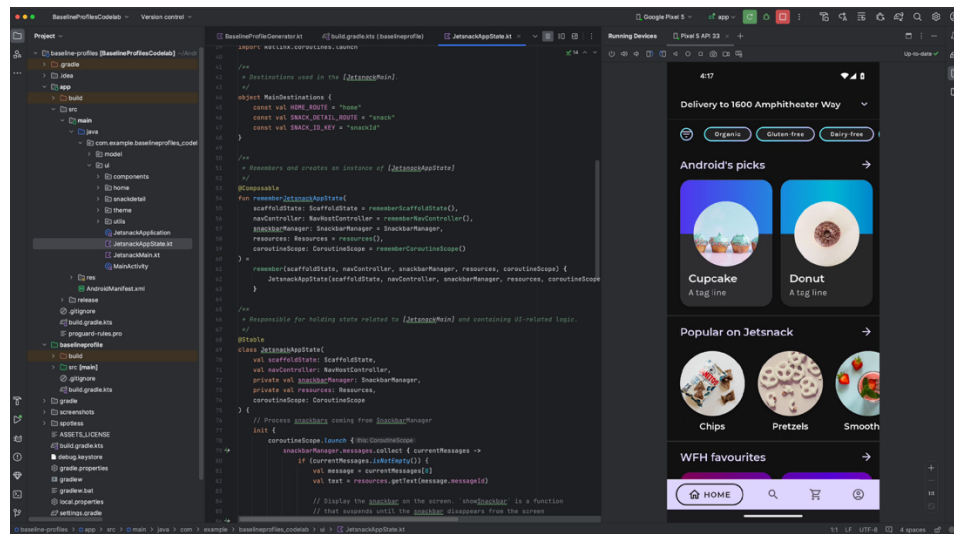


Figura 5. Captura de pantalla de Android Studio, IDE para Android, en funcionamiento.

4. Desarrollo de aplicaciones Android

La peculiaridad de los dispositivos móviles, la variabilidad que entre estos existen y, sobre todo, la multiplicidad de recursos que son capaces de aportar estos dispositivos hacen necesario que antes de iniciar las tareas de codificación se conozcan los fundamentos y la estructura de componentes en los que se basa una aplicación en Android.

Además, el correcto manejo de algunos de estos componentes (Android Manifest, clase R, permisos...) es importante si se desea que las aplicaciones desarrolladas tengan alguna proyección de mayor alcance que el límite que pueda imponer el propio equipo informático y las herramientas de desarrollo.

4.1. Componentes de una aplicación

Hay cuatro tipos distintos de **componentes**; cada uno tiene un fin específico y un ciclo de vida diferente (más adelante, serán estudiados con mayor profundidad). Para entender los fundamentos de una aplicación en Android, los primeros componentes con los que es necesario familiarizarse son los siguientes:

- **Activity:** es el principal componente de una aplicación de Android y se encarga de gestionar gran parte de las interacciones con el usuario. Representa una pantalla independiente con una interfaz de usuario. Una aplicación puede tener varias actividades que, aunque independientes, colaboran entre sí en el funcionamiento de la aplicación.
- **Service:** son aplicaciones que corren de fondo para hacer operaciones de larga duración o trabajo en procesos remotos. Un servicio no tiene interfaz de usuario. Una actividad puede iniciar el servicio y permitir que se ejecute o enlazarse con él para desarrollar su cometido.
- **Content Provider:** se ocupa de gestionar un conjunto de datos de una aplicación para compartir. A través del proveedor de contenido, otras aplicaciones pueden consultar o incluso modificar información (si el proveedor lo permite).
- **Broadcast Receiver:** es una utilidad de Android que permite responder a anuncios broadcast (difusión) del sistema. Frecuentemente, un receptor de mensajes es simplemente un enlace con otros componentes realizando una cantidad mínima de trabajo.

4.2. Android Manifest

Para incluir algunos de los anteriores componentes en las aplicaciones, será necesario que el sistema reconozca la existencia de ese componente, y eso lo hará leyendo el archivo **Manifest** (*AndroidManifest.xml*) de la APP. El archivo de manifiesto proporciona información esencial sobre la aplicación al sistema Android, información que el sistema debe tener para poder ejecutar el código de la APP. **Todas las aplicaciones tienen este archivo**, cuya gestión se puede realizar a través del entorno de desarrollo; una aplicación debe tener declarados todos sus componentes en este archivo.

Su principal tarea es informar al sistema acerca de los componentes de la aplicación. Otras funciones de este archivo serán registrar los permisos asociados a la aplicación, librerías que utiliza, declarar las necesidades de software/hardware, así como el nivel de API mínimo que requiere la aplicación. Este archivo está escrito en lenguaje de marcas y tiene un aspecto similar al que se puede ver a continuación:

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- Configuración global de la aplicación -->
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.HelloWorldApp">

        <!-- Declaración de la actividad principal -->
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name">

            <!-- El Intent Filter define cómo y cuándo se inicia este componente -->
            <intent-filter>
                <!-- Indica que esta actividad es el punto de entrada principal -->
                <action android:name="android.intent.action.MAIN" />

                <!-- Indica que debe aparecer en el lanzador (pantalla de inicio) del dispositivo -->
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>
    </application>
</manifest>
```

4.3. Recursos de una aplicación

Este término hace referencia a los **datos** que utiliza la aplicación, entendiendo como tal las imágenes, textos, estilos. Estos, junto con el código, configuran una aplicación.

Los recursos pueden estar predeterminados para la aplicación que se haya programado y se usarán siempre sin importar el tipo o la configuración del dispositivo sobre el que se vaya a ejecutar. O bien pueden crearse recursos alternativos, que son aquellos que se diseñan para usar con una configuración específica de dispositivo (tablet, pantallas grandes...), colaborando, en parte, con la responsividad de la aplicación.

Además, los recursos pueden estar gestionados por ficheros XML, deben tener asignado un **identificador** (*id*) y suelen estar alojados en una subcarpeta dentro de la carpeta principal denominada **res**. Los nombres de estas subcarpetas son limitados y predefinidos por Android. Las subcarpetas de **res** permiten establecer una organización de los recursos:

- **drawable**: recursos dibujables, ficheros de bitmaps o de imágenes "escalables". También pueden definirse aquí formas y colores de objetos dibujados.
- **layout**: definidos como ficheros XML. Engloban los elementos visuales que definen el interfaz de nuestra aplicación.

- **animator**: permite crear animaciones sencillas sobre uno o varios gráficos, como rotaciones, fading, movimiento y estiramiento. Cada animación se define en un fichero XML.
- **mipmap**: con contenido similar a drawable, el directorio mipmap alberga elementos bitmap, aunque se suele utilizar específicamente para ubicar el icono de la aplicación.
- **menu**: son ficheros XML donde se definen las diferentes opciones de los menús, submenús, barras de navegación incluidos en las aplicaciones.
- **values**: es una carpeta con carácter genérico con ficheros XML que configuran diferentes aspectos de la aplicación, como es el color, dimensiones, cadenas de texto, estilos...
- **xml**: almacena archivos XML utilizados por Android para dar soporte a tareas múltiples, como la que permite configurar búsquedas.
- **raw**: se usa para almacenar los archivos raw de la aplicación (audio y vídeo). Este directorio tiene ciertas limitaciones en cuanto al nombre de archivos que puede tener (mayúsculas, números, caracteres especiales), por lo que, si se necesita usar el nombre real del archivo, entonces se deben colocar los archivos raw en el directorio *asset* de Android.

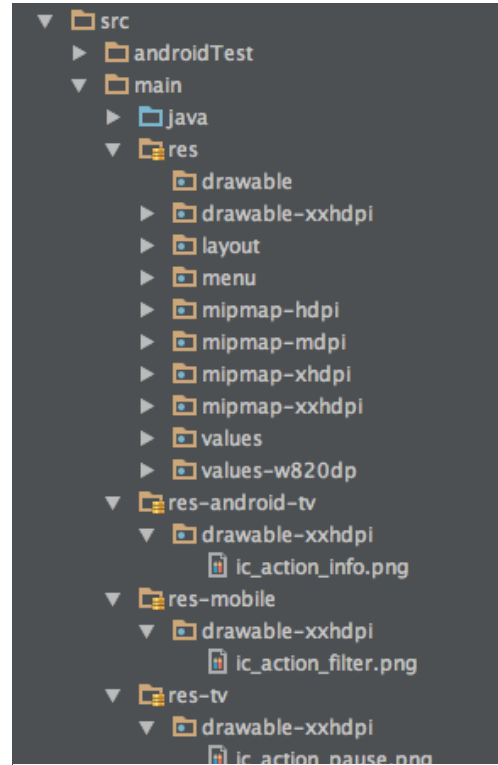


Figura 6. Carpeta res y sus subcarpetas.

4.4. La clase R

La clase **R** contiene variables estáticas en las que se identifica cada tipo de recurso. Android lee el fichero XML, carga todas las estructuras solicitadas en memoria y mantiene el fichero R como referencia directa a los recursos cargados. Por tanto, cada atributo tiene una dirección de memoria asociada referenciada a un recurso en específico.

Cuando se trabaja con código Kotlin, R se manifiesta como una estructura que agrupa distintas clases estáticas internas correspondientes a cada tipología de recurso (tales como `R.string`, `R.layout`, `R.drawable` o `R.id`). En lugar de contener los datos multimedia o los textos en sí, estas clases albergan **identificadores numéricos** enteros únicos (Int), los cuales funcionan a modo de claves de consulta dentro de una tabla de recursos empaquetada junto con la aplicación.

En tiempo de ejecución, se utilizan estas constantes enteras (por ejemplo, al invocar `getString(R.string.app_name)` o `painterResource(id = R.drawable.icono)`) para que el framework de Android localice y cargue en memoria únicamente el elemento requerido bajo demanda.

4.5. Actividades

La **actividad** (*Activity*) es el principal componente de una aplicación de Android y se encarga de gestionar gran parte de las interacciones con el usuario. A nivel de lenguaje de programación, una actividad hereda de la clase `Activity` y se traduce en una pantalla. Por tanto, y en términos generales, una aplicación de Android tendrá tantas *Activities* como pantallas distintas tenga.

Una aplicación generalmente consiste en múltiples actividades vinculadas de forma flexible entre sí. En el diseño de una aplicación, las *Activities* tendrán una secuencia de aparición según la lógica con la que se haya construido la misma. Aparecerán una u otras pantallas según el flujo del programa y las decisiones que en cada caso tome el usuario.

Desde una pantalla principal (main), se podrán realizar desplazamientos de una a otra *Activity* y, conforme se va "navegando" por las diferentes pantallas de la aplicación, el dispositivo va recordando la secuencia "visitada", de tal manera que si se retrocede utilizando los botones del dispositivo o la lógica de la aplicación, irán apareciendo pantallas en sentido inverso a como se habían visionado. Android almacena la posición de cada una de las *Activities* en una **pila** (llamada *Back Stack*).



Figura 7. Pila de actividades en Android.

El sistema de funcionamiento de esta pila es muy simple: cuando la actividad en curso inicia otra, esta nueva actividad obtiene el foco y es empujada a la parte superior de la pila; la actividad anterior permanece en la pila, pero detenida. El conjunto de actividades que tienen una relación lógica y que se encuentran en la pila se denomina **tarea** (*Task*).

Una tarea puede moverse en conjunto a un "segundo plano" cuando el usuario comienza una nueva tarea o va a la pantalla Inicio. Si está en segundo plano, todas las actividades de la tarea se paran, pero el Back Stack de esta tarea se conserva. Si una tarea vuelve a situarse en "primer plano", esta se volvería a posicionar en el lugar donde se había dejado.

4.5.1. Ciclo de vida de una actividad

Una actividad puede estar en diferentes periodos de funcionamiento denominados **estados**, y que van a depender del flujo de la aplicación, de la interacción con el usuario y de las necesidades de recursos del sistema.

La actividad debe relacionarse con el sistema operativo y con otros programas que pueden estar en diferentes estados en el dispositivo móvil. Teniendo en cuenta el estado de funcionamiento de la misma, para Android, la actividad puede ser:

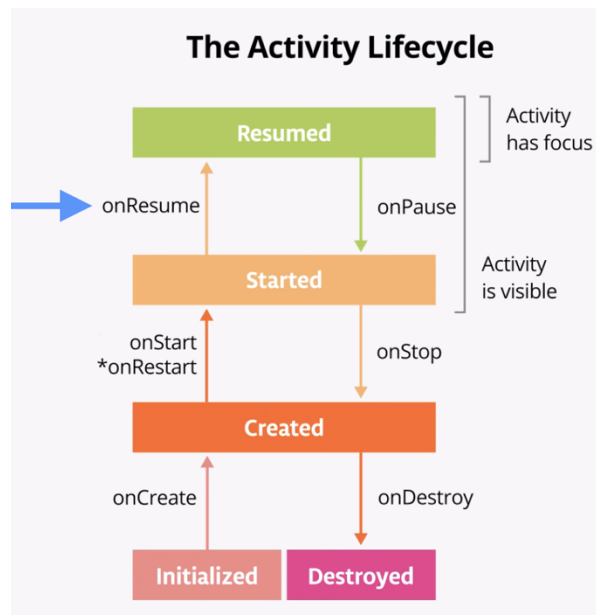
- **Nueva.** No ocupa espacio en memoria por lo que no se puede interactuar con ella.
- **Detenida.** Ocupa espacio en memoria, aunque no es visible y, por tanto, tampoco es accesible.
- **Pausada.** Es visible, pero no se puede interactuar con ella.
- **Activa.** Está en primer plano del dispositivo y el usuario puede interrelacionarse con ella.

- **Eliminada.** Se borra la información de la actividad disponible en memoria, impidiendo interacciones.

Estos estados suelen ir en sucesión creciente cuando la actividad se pone en marcha, y en sentido decreciente cuando desaparece y se destruye, por lo que el conjunto de estados en el que se puede encontrar una actividad es comúnmente denominado **ciclo de vida**.

Los cambios de estado pueden ser controlados a través de métodos, llamados por Android *callback*.

- **onCreate():** se llama cuando la actividad es invocada por primera vez. Es el momento en que se crean las vistas (la interfaz de usuario), se hace una reserva de memoria para los datos, se obtienen las referencias e instancias de componentes mediante `findViewById()`, se hacen las consultas iniciales en las fuentes de datos y se guardan las variables del entorno.



- **onStart():** este método sucede a `onCreate()`. La actividad se hace visible para el usuario, pero aún no se puede interactuar con ella. Suele ser el lugar donde ubicar instrucciones asociadas al interfaz de usuario.
- **onResume():** se llama cuando la actividad pasa a estar disponible para interactuar con el usuario. Sucede a `onStart()` y precede a `onPause()`. En este periodo se suelen activar sensores, cámara, ejecutar animaciones, actualizar información...
- **onPause():** ocurre cuando la actividad pasa a un segundo plano, bien porque está en proceso de destrucción o porque otra actividad la sustituye en el primer plano. En este periodo la actividad es parcialmente visible, y es cuando se detienen procesos, animaciones... Puede ocurrir que vuelva a primer plano (`onResume()`) o que sea parada `onStop()`, haciéndose invisible para el usuario.
- **onStop():** este método se invoca cuando la actividad ya no es visible al usuario y otra actividad ha pasado a primer plano. Se pausan animaciones, determinados

servicios se detienen (como el GPS) y se minimizan los recursos consumidos por la actividad, aunque aún conserva datos en memoria. Si se desea reactivarla, se utiliza `onRestart()`, volviendo a primer plano; u `onDestroy()`, si queremos eliminarla definitivamente.

- **`onRestart()`**: se llama cuando una actividad parada vuelve a primer plano. Siempre lo sigue un `onStart()`.
- **`onDestroy()`**: es llamado cuando la actividad es definitivamente destruida y eliminada de memoria. Suele emplearse para limpiar y liberar recursos.

Se concluye, pues, que el **ciclo de vida completo** de una actividad transcurre entre la llamada a `onCreate()` y la llamada a `onDestroy()`.

El **ciclo de vida visible** de una actividad es aquel en que el usuario puede ver la actividad en pantalla y que transcurre entre la llamada a `onStart()` y la llamada a `onStop()`.

Y finalmente, el **ciclo de vida en primer plano** de una actividad transcurre entre la llamada a `onResume()` y la llamada a `onPause()`. Es el momento que la actividad tiene el foco en el dispositivo y se encuentra por encima de todas las demás actividades que están activas en memoria.

4.6. Intents y filtros

Un ***Intent*** es un sistema de comunicación que permite interactuar entre componentes de la aplicación o de distintas aplicaciones de Android. Es un elemento básico de comunicación, tal que con él se puede comenzar una aplicación, iniciar un servicio, entregar un mensaje.

Mediante un Intent se puede enviar o recibir información desde y hacia otras actividades o servicios, así como transmitir difusiones notificando eventos del sistema o de la propia aplicación a receptores registrados. Es una manera útil de reutilizar otros componentes que ya desarrollan la funcionalidad en el código, pudiendo, por ejemplo, iniciar una aplicación que nos permita tomar fotografías sin tener que realizar el código de dicha aplicación.

Por sus características y construcción, Android dispone de dos tipos de Intents, explícitos o implícitos.

4.6.1. Intent explícito

Los **Intents explícitos** se utilizan cuando se conoce con exactitud el componente de destino que debe ejecutarse, el cual habitualmente pertenece al propio paquete de la aplicación. Su cometido principal es estructurar la navegación interna entre las diferentes actividades o invocar servicios específicos, garantizando que ninguna otra aplicación del sistema intercepte la petición.

Además, también actúan como canal de transporte de información entre componentes mediante **extras**, que son gestionados a través de contenedores tipo *Bundle*. Por medio del método `putExtra()` y las funciones de extensión de Kotlin, es posible transferir tipos primitivos, cadenas de texto o estructuras de datos complejas serializadas.

```
// Actividad origen (MainActivity.kt)
val intentExplicito = Intent(this, UsuarioActivity::class.java).apply{
    putExtra("EXTRA_USUARIO_ID", 777)
    putExtra("EXTRA_NOMBRE", "Alejandro Monfort")
}
startActivity(intentExplicito)

//Actividad destino (UsuarioActivity.kt)
val idUsuario = intent.getIntExtra("EXTRA_USUARIO_ID", -1)
val nombreUsuario = intent.getStringExtra("EXTRA_NOMBRE")
```

4.6.2. Intent implícitos

Los **Intents implícitos** no definen un componente concreto de destino, sino que declaran una **acción abstracta** que se desea llevar a cabo (como visualizar una página web, compartir un texto, realizar una llamada o abrir un mapa) junto con los datos necesarios para ejecutar la acción. Al lanzarse, Android evalúa los filtros declarados (<intent-filter>) en todas las aplicaciones instaladas en el dispositivo para determinar cuáles son capaces de responder la petición, mostrando al usuario un diálogo selector si existe más de una opción disponible.

Este mecanismo fomenta la reutilización del ecosistema móvil y desacopla a la aplicación de responsabilidades ajenas: en lugar de implementar un navegador web o un cliente de mensajería propio, se delega la tarea en las aplicaciones ya existentes. Si fuera necesario

delegar tareas para obtener un resultado de vuelta (como seleccionar un archivo o capturar una foto), se emplea la **Activity Result API** (con el método `registerForActivityResult()`), que permite asegurar la integridad del estado si el sistema destruye la actividad en segundo plano.

```
val texto = "Solicitud de texto"

val sendIntent = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_TEXT, texto)
}

// Se envuelve en un chooser para asegurar que siempre pregunte qué app usar
val chooserIntent = Intent.createChooser(sendIntent, "Compartir mensaje mediante:")

if (sendIntent.resolveActivity(packageManager) != null) {
    startActivity(chooserIntent)
}
```

4.6.3. Elementos de un Intent

El constructor de un Intent contiene diferentes elementos que permiten establecer los parámetros del mismo:

- **Nombre del Componente.** Este elemento determina de manera unívoca la clase exacta y el paquete del componente de destino (Activity, Service o BroadcastReceiver) mediante un objeto `ComponentName`. Su inclusión define si el Intent es explícito o implícito.
- **Acción.** Es una cadena de texto que describe de forma abstracta la operación general que se desea invocar en el sistema o, en el caso de las difusiones, el evento ocurrido. Se suelen emplear constantes estándar del framework, como `Intent.ACTION_VIEW` para mostrar contenidos, `Intent.ACTION_SEND` para compartir información o `Intent.ACTION_DIAL` para abrir el marcador telefónico.
- **Datos y Tipo MIME.** Estos dos campos determinan de manera conjunta sobre qué contenido se va a operar y cuál es su formato específico. El campo *data* almacena una dirección URI (`android.net.Uri`), como puede ser un esquema web, un número de teléfono o una posición geográfica. El campo *type* especifica el tipo MIME del contenido, como *image/png* o *text/plain*.

- **Categoría.** Es una cadena de texto complementaria que califica y restringe el tipo de entorno o componente que debe admitir la petición. Se gestionan mediante la función `addCategory()`.
- **Datos extra.** Los extras constituyen una estructura de tipo clave-valor implementada a través de un *Bundle*, diseñada para transportar información complementaria que la actividad o servicio de destino necesita para ejecutar su lógica. Se adjuntan utilizando funciones de extensión o el método sobrecargado `putExtra()`, admitiendo diversos tipos de datos como se ha explicado previamente.
- **Indicadores.** Son constantes enteras binarias definidas en la clase `Intent` que actúan a modo de directivas de bajo nivel para instruir al sistema operativo sobre cómo debe procesarse la petición. Se asignan modificando la propiedad `flags` o invocando `addFlags()`, y su propósito habitual es alterar la gestión de la pila de tareas.

Intent explícito

```
val nombreIntent = Intent(contextoOrigen, ClaseDestino::class.java).apply {  
    putExtra("CLAVE_DATO_1", valorDato1)  
    putExtra("CLAVE_DATO_2", valorDato2)  
    addFlags(Intent.FLAG_ACTIVIDAD_DESEADA)  
}  
startActivity(nombreIntent)
```

Intent implícito

```
val nombreIntent = Intent(Intent.ACTION_NOMBRE_ACCION).apply {  
    data = Uri.parse("esquema:recurso")  
    type = "tipo/mime"  
    putExtra(Intent.EXTRA_TEXTO_OPCIONAL, valor)  
    addCategory(Intent.CATEGORY_NOMBRE_CATEGORIA)  
}  
startActivity(nombreIntent)
```